

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI - 590 018, KARNATAKA



An Devops Project Report on
**“ VersionIQ: AI-Powered Version Intelligence and DevOps
Automation Platform ”**

By

Ms. SUPRIYA GOUDA

USN: 1BY23IS227

Ms. VARSHA

USN: 1BY23IS246

Ms. VIDHI PODDAR

USN: 1BY23IS248

Under the guidance of

Prof. Amulya P
Assistant Professor
Department of ISE, BMSIT&M.



DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING
BMS INSTITUTE OF TECHNOLOGY & MANAGEMNT
Yelahanka, Bengaluru-560 119
2025-26

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI – 590 018, KARNATAKA

BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT

YELAHANKA, BENGALURU-560 119

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Project work entitled “**VersionIQ: AI-Powered Version Intelligence and DevOps Automation Platform**” is a bonafide work carried out by **Ms. Supriya Gouda (1BY23IS227)**, **Ms. Varsha (1BY23IS246)** and **Ms. Vidhi Poddar (1BY23IS248)**, as Alternate Assessment Tool (AAT), in partial fulfillment of the course DevOps (BCS608C) for the award of **Bachelor of Engineering Degree In Information Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2026-27. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in this report. The project report has been approved as it satisfies the academic requirements.

Signature of the Guide
(Prof. Amulya P)

Signature of the Cluster Head
(Dr. N. Rakesh)

Signature of the HOD
(Dr. Surekha K B)

ACKNOWLEDGEMENT

We are happy to present this project (AAT) after completing it successfully. This project would not have been possible without the guidance, assistance and suggestions of many individuals. We would like to express our deep sense of gratitude and indebtedness to each and every one who has helped us make this mini project a success.

We heartly thank **Dr. Sanjay H A, Principal, BMS Institute of Technology & Management** for his constant encouragement and inspiration in taking up this project.

We heartly thank **Dr. Surekha K.B., Head of the Department, Professor & Head, Department of Information Science and Engineering, BMS Institute of Technology & Management** for his constant encouragement and support in taking up this project.

We heartly thank **Dr .N. Rakesh, Associate Head of ISE Cluster-5 , Information Science and Engineering, BMS Institute of Technology & Management** for constant encouragement and inspiration in taking up this project.

We gracefully thank our Project guide, **Prof. Amulya P, Assistant Professor, Department of Information Science and Engineering, BMS Institute of Technology & Management** for her encouragement and guidance throughout the course of the project work.

Special thanks to all the faculty and staff of Dept. of Information Science and Engineering for their help and kind co-operation.

We also thank our parents and friends for their unconditional love and encouragement and support given to us in order to finish this precious work.

Last but not the least we would like to thank God for giving us the strength and motivation through the course of this Project.

By,

Supriya Gouda

Varsha

Vidhi Poddar

ABSTRACT

VersionIQ is an AI-powered Version Intelligence and DevOps Automation platform designed to manage file uploads, version history, semantic change analysis, rollback recommendations, and CI/CD workflow monitoring within a single integrated system. Traditional version management systems mainly focus on storing file history and tracking line-based modifications, making it difficult for developers to understand the actual purpose and impact of changes. To overcome these limitations, the proposed system uses Artificial Intelligence and DevOps technologies to provide intelligent version analysis, automated deployment workflows, and real-time monitoring capabilities. The platform is developed using React with TypeScript for the frontend and Node.js with Express.js for backend services, while MongoDB is used for storing user information, file metadata, version records, AI summaries, rollback logs, and deployment details.

The system integrates Google Gemini AI to analyze differences between file versions and generate human-readable semantic summaries explaining added, removed, and modified changes. Multer middleware is used for secure file upload and storage operations. Docker is used for containerization to ensure consistent deployment environments, while Jenkins automates Continuous Integration and Continuous Deployment workflows through CI/CD pipelines. The DevOps dashboard provides real-time visibility into deployment status, build activity, pipeline execution, rollback recommendations, and container health monitoring. Overall, VersionIQ combines Artificial Intelligence, full stack development, Docker containerization, and DevOps automation into a unified platform for intelligent version management and deployment monitoring.

CONTENTS

<i>Acknowledgement</i>	<i>i</i>
<i>Abstract</i>	<i>ii</i>
<i>List of Figures</i>	<i>iv</i>
Chapter	Page No.
Chapter 1: INTRODUCTION	1 – 2
1.1 Overview	1
1.2 Summary	2
Chapter 2: LITERATURE SURVEY	3 - 6
2.1 Review of Existing Systems	3
2.2 Motivation	4
2.3 Problem Statement	4
2.4 Objectives	5
2.5 Methodology	6
Chapter 3: DESIGN AND IMPLEMENTATION	7 - 14
3.1 System Architecture and Design	7
3.2 Technical Implementation	9
3.3 Results	10
Chapter 4: DEVOPS PRACTICES	15 - 16
4.1 Version Control and Collaboration (GitHub)	15
4.2 Build Automation and Dependency Management (Jenk-ins and CI/CD)	16
4.3 Environment Containerization (Docker)	16
Chapter 5: CONCLUSION	17 - 18
5.1 Advantages	17
5.2 Disadvantages	17
5.3 Future Scope	18

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	System Architecture of VersionIQ	9
3.2	Landing Page	11
3.3	Dashboard Page	11
3.4	AI Summary of the versions	12
3.5	Smart Rollback of the versions created	12
3.6	The overview of the latest build of the CI / CD Pipeline	13
3.7	Jenkins Setup and Build	13
3.8	Docker Setup and Running page	14
3.9	GitHub Webhook configuration and delivery	14

Chapter 1

INTRODUCTION

1.1 Overview

VersionIQ: AI-Powered Version Intelligence and DevOps Automation Platform is a modern full stack web application developed to simplify file version management, semantic change analysis, rollback support, and CI/CD workflow monitoring. In traditional version management systems, users often struggle to understand file modifications because most systems only provide raw line-by-line differences without explaining the actual purpose or impact of the changes. Manual analysis of file differences, deployment failures, and rollback operations becomes difficult and time-consuming, especially in large-scale software development environments. The proposed system provides an intelligent and centralized platform that combines Artificial Intelligence, version management, and DevOps automation to improve workflow efficiency, deployment visibility, and software reliability.

The platform is developed using React with TypeScript for the frontend and Node.js with Express.js for backend services. MongoDB is used for storing user information, file metadata, version history, rollback records, AI summaries, and deployment logs. The system integrates Google Gemini AI to analyze differences between file versions and generate human-readable semantic summaries explaining added, removed, and modified changes. Docker is used for containerization, while Jenkins automates Continuous Integration and Continuous Deployment workflows through CI/CD pipelines. By combining AI-powered version intelligence with DevOps automation, VersionIQ provides a scalable, secure, and intelligent platform for modern software development and deployment management.

1.2 Summary

The proposed VersionIQ platform is an AI-powered full stack application developed to modernize and simplify version management, deployment monitoring, and rollback analysis. The system allows users to securely upload files, maintain version history, compare modifications, and generate AI-powered semantic summaries explaining differences between file versions. The application also integrates Jenkins and Docker to automate build, testing, deployment, and monitoring operations through CI/CD pipelines. MongoDB is used for reliable and secure data storage, while the DevOps dashboard provides real-time visibility into deployment status, pipeline activity, rollback recommendations, and container health monitoring. Over-all, the platform demonstrates the integration of Artificial Intelligence, full stack development, Docker containerization, and DevOps automation to build a scalable and intelligent Version Intelligence and Deployment Monitoring system.

Chapter 2

LITERATURE SURVEY

2.1 Review of Existing Systems

Existing version management and deployment monitoring systems used in many software development environments mainly focus on preserving source code history and tracking file modifications. Traditional version control systems such as Git and GitHub allow developers to maintain repositories, manage commits, and collaborate through branches. However, these systems primarily provide raw line-by-line differences between versions and do not generate meaningful semantic explanations of changes. Developers often spend significant time manually analyzing commits, understanding deployment-related modifications, and identifying the impact of changes on system stability. Many existing systems also lack integrated rollback intelligence, centralized monitoring dashboards, and AI-powered semantic analysis capabilities.

Several cloud storage and file versioning systems preserve previous versions of files but generally focus only on storage and retrieval functionalities. These systems do not provide intelligent analysis of version differences, deployment-aware rollback recommendations, or integrated DevOps automation. Existing deployment management solutions often require separate tools for version control, CI/CD pipelines, monitoring, and rollback operations, resulting in fragmented workflows and increased operational complexity. In addition, many traditional systems do not support AI-generated summaries, containerized deployment environments, or real-time CI/CD monitoring. These limitations highlight the need for a modern and intelligent Version Intelligence and DevOps Automation platform capable of combining semantic analysis, rollback support, deployment automation, and centralized monitoring within a single system.

2.2 Motivation

The motivation behind developing VersionIQ is to overcome the limitations of traditional version control and deployment management systems by providing an intelligent, automated, and centralized platform for version tracking and DevOps monitoring. Manual analysis of file differences and deployment failures becomes difficult and time-consuming in large-scale software projects. Developers often struggle to understand the actual purpose and impact of modifications, especially when changes occur frequently across multiple files and deployment environments. To address these challenges, the project aims to integrate Artificial Intelligence with version management and DevOps automation to improve code understanding, deployment reliability, and workflow efficiency.

The project is also motivated by the increasing demand for modern DevOps practices such as Continuous Integration and Continuous Deployment (CI/CD), containerized deployment environments, and automated monitoring systems. Technologies such as Docker, Jenkins, GitHub, and the Google Gemini API provide opportunities to automate deployment pipelines, analyze file changes semantically, and simplify rollback operations. Additionally, the project provides practical exposure to real-world full stack development and DevOps workflows using React, Node.js, MongoDB, Docker, and Jenkins, which are essential technologies in modern software engineering industries.

2.3 Problem Statement

In modern software development and deployment environments, files and source code are continuously modified, updated, and deployed across multiple systems. Traditional version control systems mainly focus on storing version history and tracking line-based modifications but fail to provide meaningful insights into the purpose and impact of changes. Developers generally rely on manual comparison methods and raw file differences to understand modifications, making the process time-consuming and inefficient for large files, deployment scripts, and continuously evolving project assets. Existing systems also lack intelligent rollback recommendations, centralized deployment monitoring, and AI-powered semantic analysis features.

In deployment workflows, even small configuration changes may result in build failures, unstable deployments, or application downtime. Without proper deployment intelligence and automated monitoring systems, identifying the root cause of failures becomes difficult and increases operational complexity. Existing platforms generally treat version management, deployment automation, rollback handling, and monitoring as separate functionalities, requiring users to depend on multiple tools for complete workflow management. Therefore, there is a need for a modern Version Intelligence and DevOps Automation platform that can automatically analyze version changes, generate AI-powered semantic summaries, provide rollback intelligence, and integrate Docker-based deployment with Jenkins CI/CD automation into a unified full stack system.

2.4 Objectives

- To develop a secure full stack web application for intelligent file upload, storage, and version management.
- To generate AI-powered semantic summaries that explain differences between file versions in human-readable form.
- To implement file comparison and diff analysis for identifying added, removed, and modified content between versions.
- To develop a rollback recommendation system that helps users restore stable previous versions during deployment failures.
- To integrate Jenkins CI/CD pipelines for automating build, testing, and deployment workflows.
- To use Docker containerization for creating consistent, scalable, and isolated deployment environments.
- To provide real-time DevOps monitoring dashboards for tracking pipeline status, deployment logs, and container health.
- To implement secure authentication and centralized monitoring using modern full stack technologies such as React, Node.js, Express.js, and MongoDB.

2.5 Methodology

1. **Docker:** Docker is used to containerize the VersionIQ platform and its supporting services. It helps in creating isolated and consistent execution environments for the frontend, backend, MongoDB, and Jenkins services, making deployment easier across different systems without dependency conflicts.
2. **Git and GitHub:** Git is used as the version control system for managing source code changes, maintaining project history, and enabling collaborative development. GitHub acts as the remote repository hosting platform and integrates with Jenkins to automate the CI/CD workflow.
3. **Jenkins:** Jenkins is used as the automation server for implementing Continuous Integration and Continuous Deployment workflows. It automates project building, testing, Docker image creation, deployment, and pipeline monitoring processes.
4. **CI/CD Pipeline:** The CI/CD pipeline automates the software development lifecycle by continuously building, testing, and deploying the application whenever code changes are pushed to the GitHub repository. This improves deployment reliability and reduces manual effort.
5. **Gemini AI Integration:** Google Gemini AI is integrated into the backend to analyze differences between file versions and generate semantic summaries explaining modifications in a human-readable format.
6. **MongoDB Database:** MongoDB is used to securely store user information, file metadata, version records, rollback history, activity logs, AI summaries, and deployment pipeline details.
7. **Full Stack Development:** React with TypeScript is used for frontend development, while Node.js and Express.js handle backend services, RESTful APIs, authentication, file processing, AI integration, and DevOps communication.

Chapter 3

DESIGN AND IMPLEMENTATION

3.1 System Architecture and Design

The system architecture of VersionIQ follows a modern full stack and DevOps-integrated architecture model to provide scalability, automation, maintainability, and intelligent version management. The platform combines frontend technologies, backend services, Artificial Intelligence, database management, and DevOps automation into a centralized environment for managing file versions and deployment workflows. The following is the system architecture of VersionIQ:

1. System Architecture:

The VersionIQ platform follows a client-server architecture integrated with AI services and DevOps automation tools. The architecture is designed to support secure communication, real-time monitoring, semantic version analysis, rollback intelligence, and automated deployment workflows.

2. Frontend Layer:

The frontend is developed using React with TypeScript to provide an interactive and responsive user interface. It manages file uploads, version browsing, AI summary visualization, rollback recommendations, deployment monitoring dashboards, and CI/CD pipeline analytics.

3. Backend Layer:

The backend is developed using Node.js and Express.js for handling authentication, RESTful APIs, file processing, version creation, AI integration, rollback services, and communication with Jenkins and MongoDB. The backend also manages secure user sessions using JWT authentication.

4. AI Integration Layer:

The backend communicates with the Google Gemini API to analyze differences between current and previous file versions. The AI service generates semantic summaries describing added, removed, and modified content in a human-readable format.

5. Database Layer:

MongoDB is used for securely storing user information, file metadata, version history, AI summaries, rollback records, activity logs, deployment records, and pipeline monitoring data. Mongoose is used for schema modeling and database operations.

6. File Storage System:

Uploaded files are stored securely on the server file system using Multer middleware. The system maintains structured directories based on user ID, version number, and file information for organized version tracking.

7. DevOps and CI/CD Layer:

Jenkins is integrated to automate Continuous Integration and Continuous Deployment workflows. Docker is used to containerize the frontend, backend, database, and Jenkins services, ensuring consistent execution environments across different systems.

8. Communication Flow:

The frontend communicates with the backend through RESTful APIs. The backend interacts with MongoDB for data storage, Gemini AI for semantic analysis, and Jenkins for CI/CD pipeline monitoring and deployment synchronization.

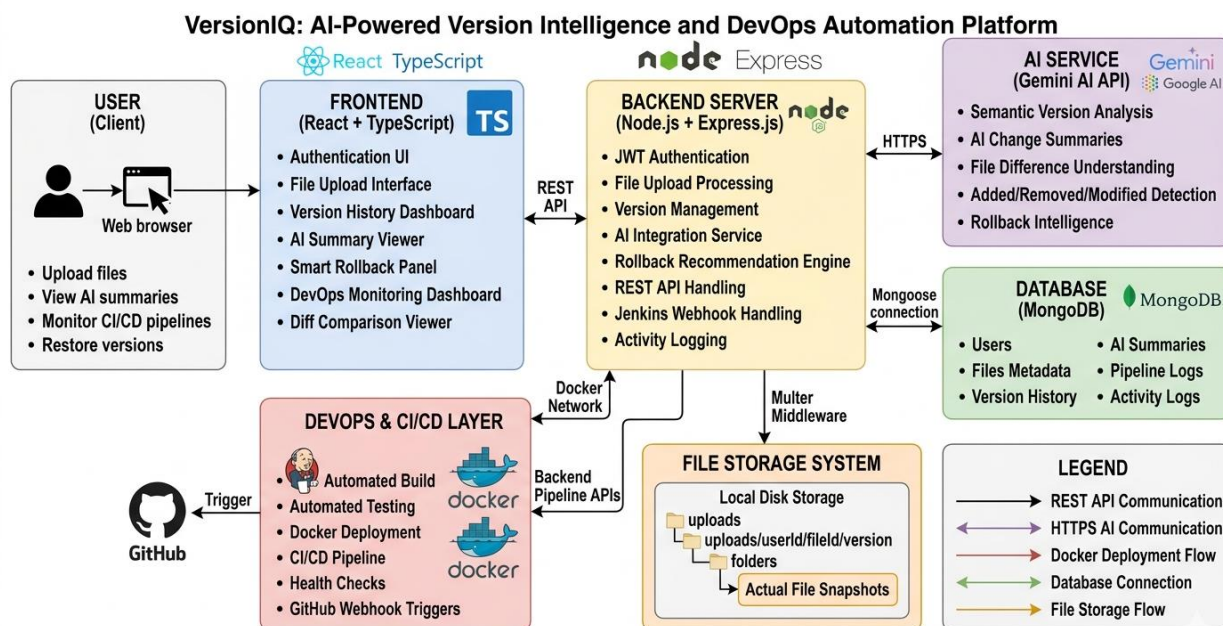


Figure 1: System Architecture of VersionIQ – AI-Powered Version Intelligence and DevOps Automation Platform

Fig 3.1 : System Architecture of VersionIQ

3.2 Technical Implementation

1. Main View:

The technical implementation of VersionIQ is carried out using the MERN stack and DevOps technologies. React with TypeScript is used to build the frontend interface, while Node.js and Express.js are used to develop backend services and RESTful APIs. The platform is divided into multiple modules such as authentication, file management, AI analysis, rollback intelligence, and DevOps monitoring to ensure modular and scalable development.

2. Database and API Integration:

MongoDB is used as the primary database for storing user details, file metadata, version history, deployment records, and AI summaries securely. Mongoose simplifies database operations such as insertion, retrieval, updating, and deletion of records. RESTful APIs are implemented for communication between frontend and backend services.

3. AI Semantic Analysis:

The backend extracts readable content from uploaded files and compares the current version with previous versions. The extracted content is sent to the Gemini AI API, which generates semantic summaries explaining file modifications and deployment-related changes in human-readable form.

4. Docker and Containerization:

Docker is used to containerize the frontend, backend, MongoDB, and Jenkins services. Docker Compose manages multiple containers together using a single configuration file, simplifying deployment and orchestration across different environments.

5. Jenkins and CI/CD Pipeline:

Jenkins automates the software development lifecycle through a CI/CD pipeline. When-ever code changes are pushed to the GitHub repository, Jenkins automatically performs source code checkout, project build, testing, Docker image creation, deployment, and health verification operations.

3.3 Results

The implementation of VersionIQ successfully demonstrated intelligent file version management, AI-powered semantic change analysis, rollback recommendations, and DevOps automation within a unified platform. The system effectively maintained file version history, generated human-readable summaries for file modifications, and provided rollback intelligence for restoring stable versions during deployment failures.

Docker containerization ensured smooth and consistent deployment across different environments, while Jenkins automation and CI/CD pipelines simplified build, testing, and deployment workflows. The integration of MongoDB provided reliable and secure storage for file metadata, version records, activity logs, and deployment monitoring details. The real-time DevOps dashboard successfully displayed pipeline status, deployment logs, and container health

information, improving deployment visibility and workflow management. Overall, the system demonstrated scalability, automation, reliability, and effective integration of Artificial Intelligence with DevOps practices.

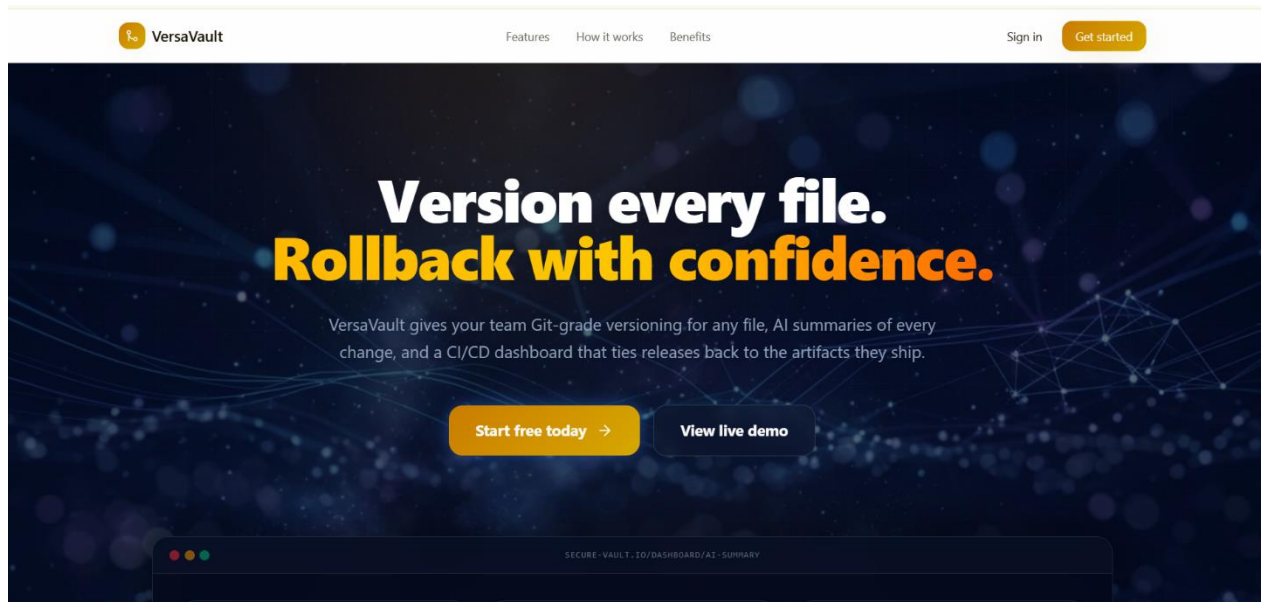


Fig 3.2 : Landing Page

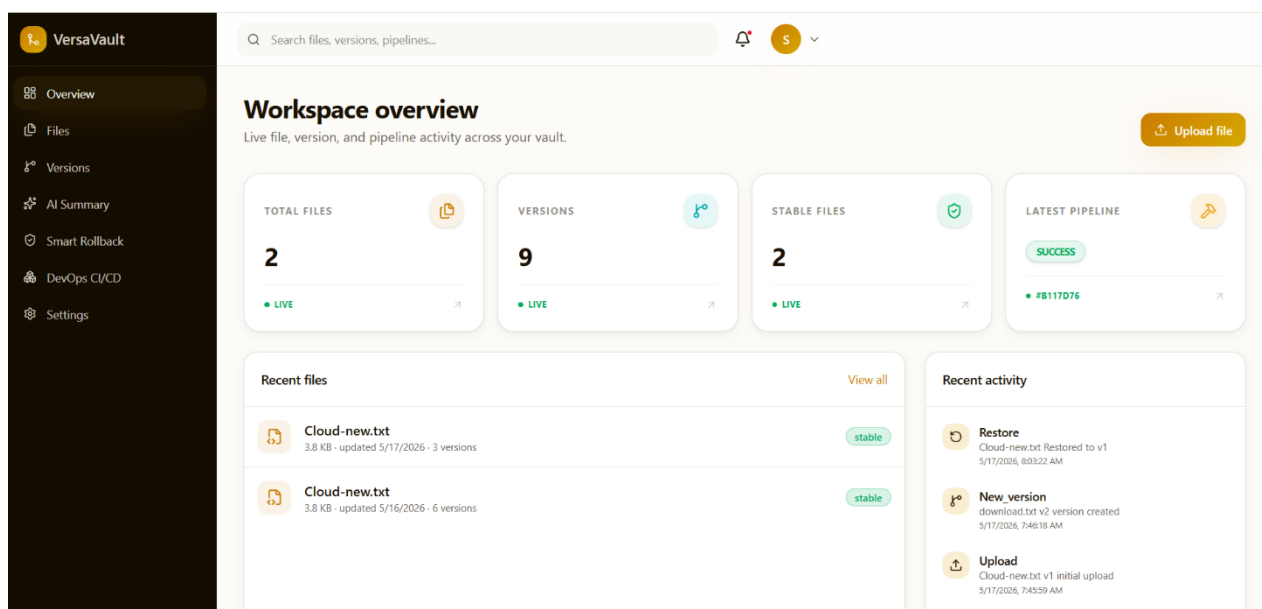


Fig 3.3 : Dashboard Page

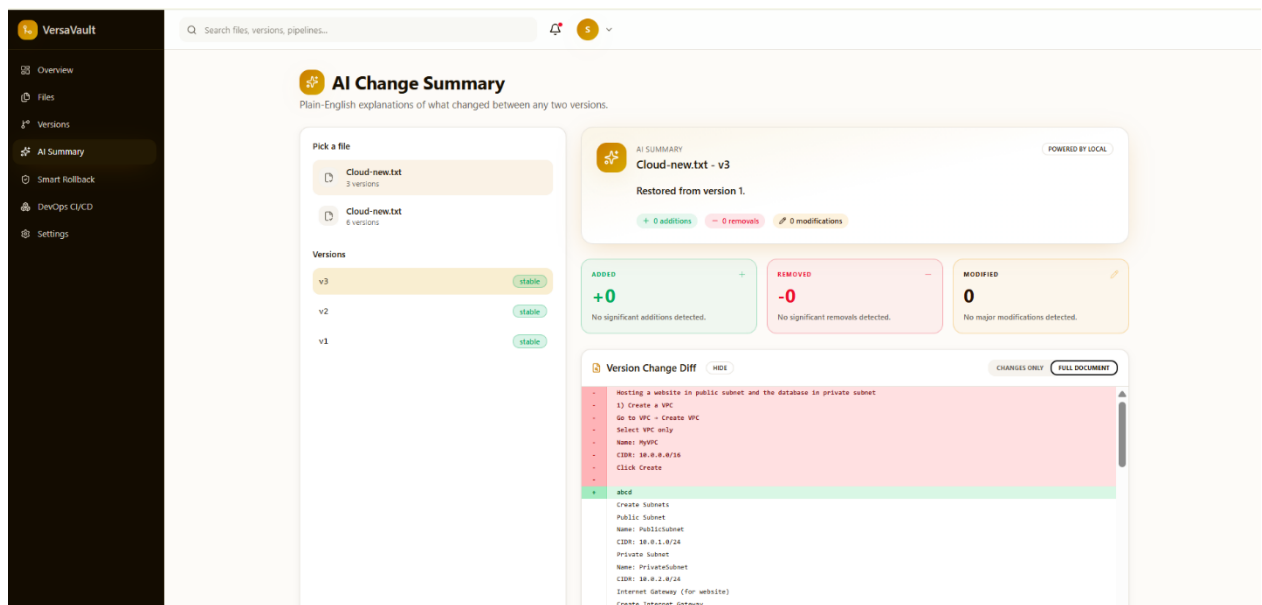


Fig 3.4 : AI Summary of the versions

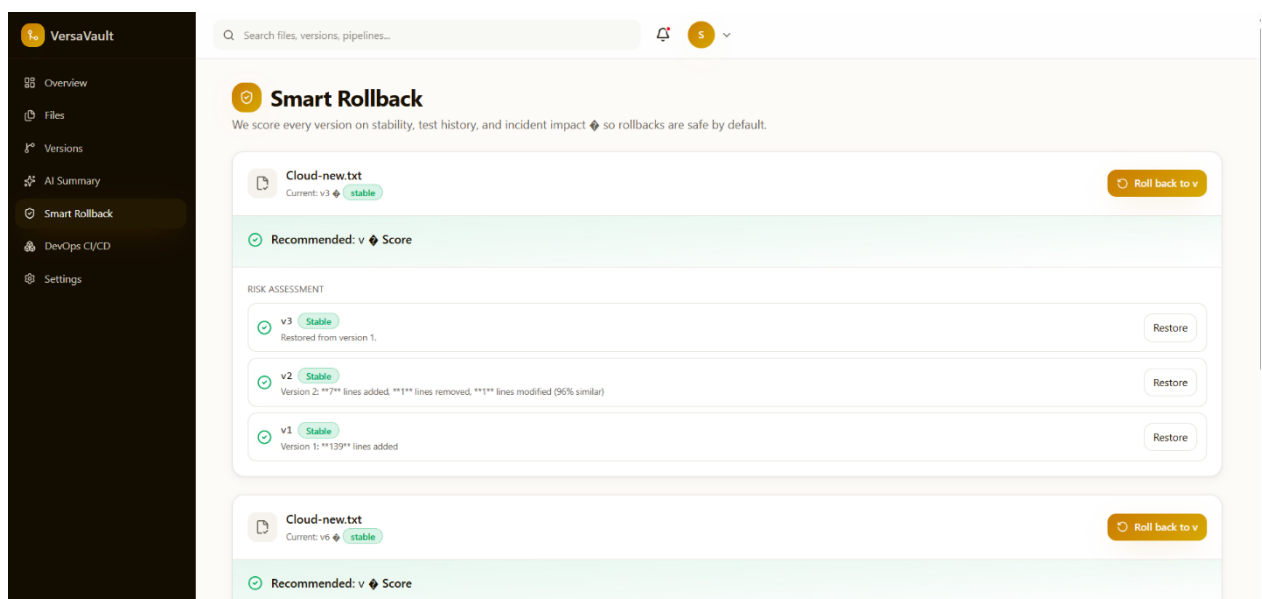


Fig 3.5 : Smart Rollback of the versions created

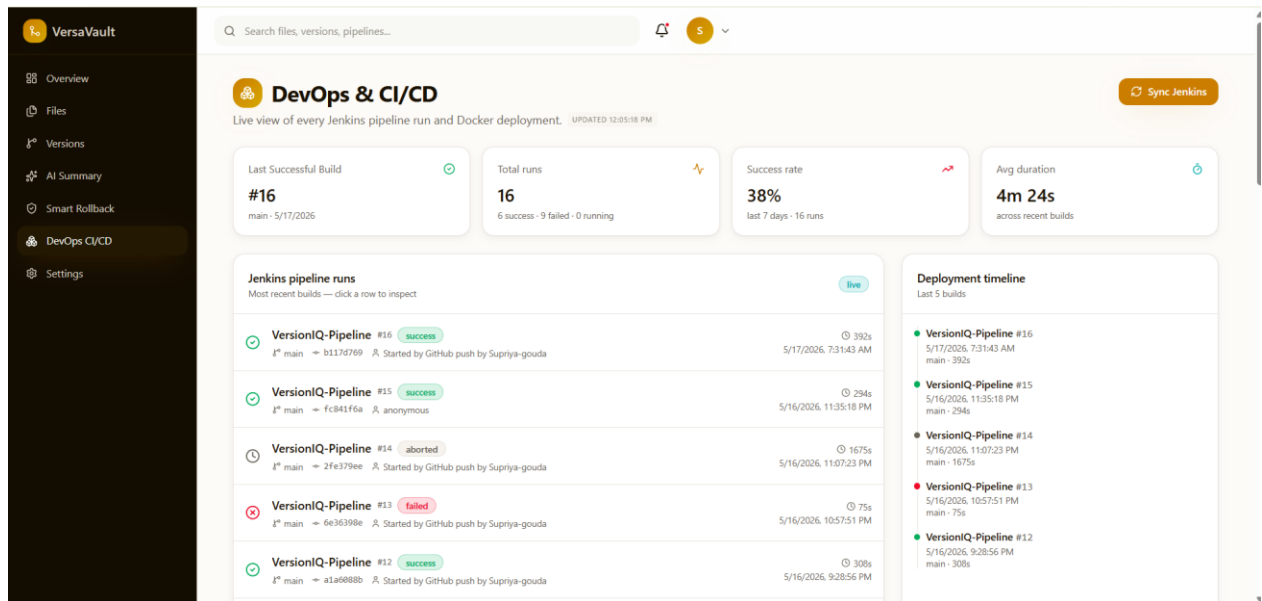


Fig 3.6 : The overview of the latest build of the CI / CD Pipeline

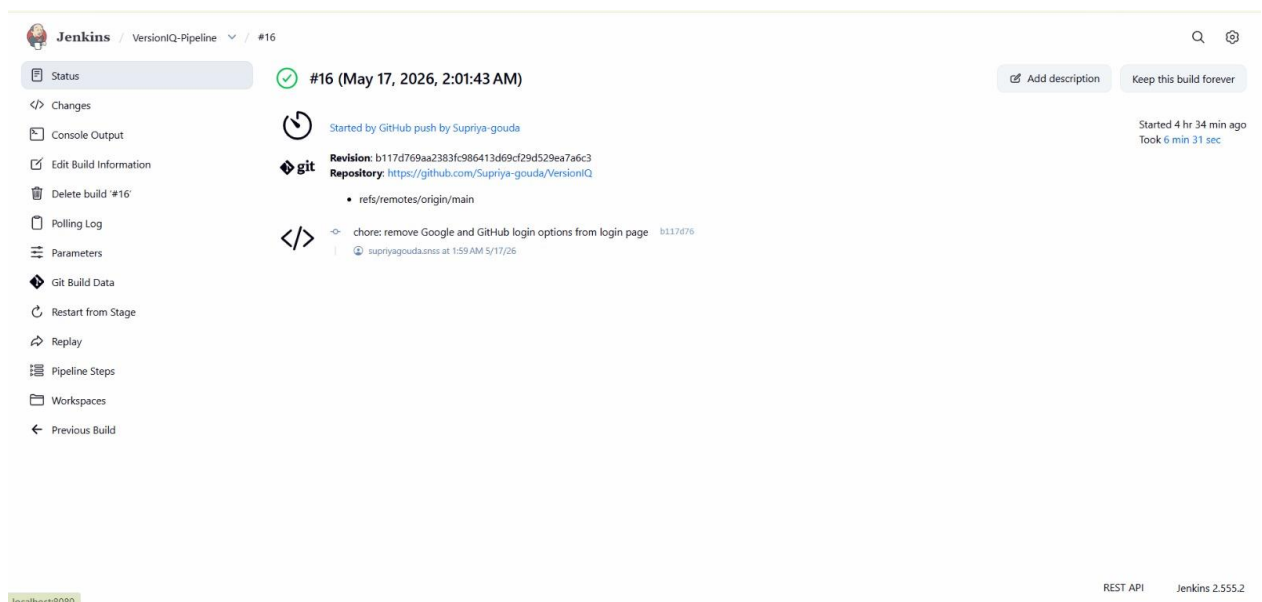


Fig 3.7 : Jenkins Setup and Build

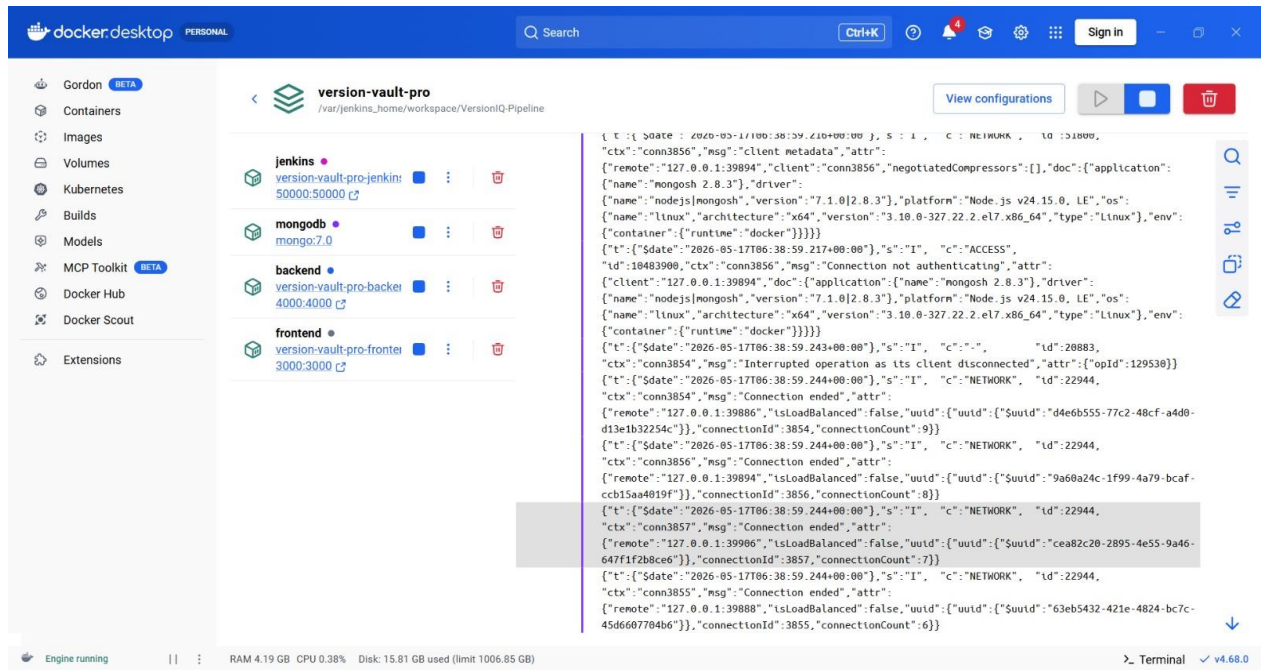


Fig 3.8 : Docker Setup and Running page

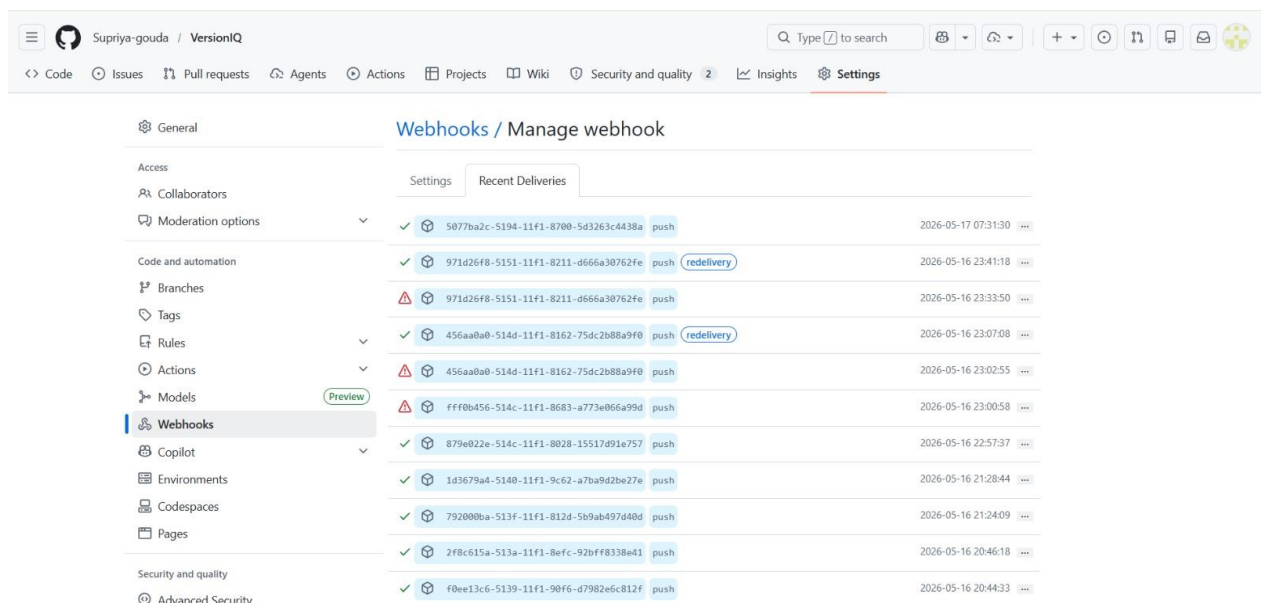


Fig 3.9 : GitHub Webhook configuration and delivery

Chapter 4

DEVOPS PRACTICES

To ensure a professional, scalable, and automated software development lifecycle, VersionIQ incorporates modern DevOps methodologies and deployment practices. Continuous Integration and Continuous Deployment (CI/CD), containerization, version control, and automated monitoring are integrated into the platform to improve deployment reliability, reduce manual effort, and maintain consistent execution environments. The project uses GitHub, Jenkins, Docker, and CI/CD pipelines to automate development and deployment workflows.

4.1 Version Control and Collaboration (GitHub)

GitHub is used in the VersionIQ project for version control, source code management, and collaborative development. Git is used to track changes made to the project files, maintain different versions of the application, and manage development activities efficiently. Using Git commands such as commit, push, pull, merge, and branch management, the development workflow becomes more organized, secure, and maintainable. Git also helps maintain the complete history of source code modifications and allows rollback to previous stable versions whenever required.

GitHub acts as the remote repository hosting platform where the project source code is securely stored and shared among team members. It enables collaborative development by allowing contributors to work on separate branches and merge changes through pull requests. The GitHub repository is connected with Jenkins to automate the CI/CD pipeline workflow. Whenever code changes are pushed to the repository, Jenkins automatically fetches the updated code and initiates the build, testing, deployment, and monitoring process. This integration improves collaboration, deployment automation, and project maintainability.

4.2 Build Automation and Dependency Management (Jenkins and CI/CD)

Jenkins is used in VersionIQ to automate build, testing, deployment, and monitoring operations through a Continuous Integration and Continuous Deployment pipeline. The automation process begins whenever source code changes are pushed to the GitHub repository. Jenkins automatically pulls the updated code, installs dependencies, executes testing scripts, builds the application, creates Docker images, and deploys the containers. This reduces manual effort, improves development speed, and helps identify errors during the early stages of development.

The CI/CD pipeline automates the entire software development lifecycle, ensuring that every update is continuously integrated, tested, and deployed without manual intervention. Jenkins also performs deployment health verification and updates the DevOps monitoring dashboard with real-time pipeline status and deployment activity. The automated pipeline improves operational consistency, deployment reliability, and software quality within the VersionIQ platform.

4.3 Environment Containerization (Docker)

Docker is used in the VersionIQ project to containerize the application and all supporting services, ensuring consistent execution across development, testing, and production environments. Containerization packages the frontend, backend, MongoDB database, and Jenkins server along with all dependencies, libraries, and configurations into isolated containers. This eliminates environment-specific issues and simplifies deployment on different systems. Docker improves portability, scalability, resource utilization, and deployment consistency.

In this project, Docker is used to create separate containers for the React frontend, Node.js backend, MongoDB database, and Jenkins automation server. Docker Compose is utilized to manage and orchestrate multiple containers together using a single configuration file. This approach simplifies application setup, networking, service dependency management, and deployment operations. Docker integration also supports the Jenkins CI/CD pipeline by enabling automated image creation, deployment, and container monitoring. Overall, Docker improves deployment efficiency, scalability, maintainability, and operational reliability within the VersionIQ platform.

Chapter 5

CONCLUSION

5.1 Advantages

- Provides intelligent file version management with complete version history tracking.
- Generates AI-powered semantic summaries that help users understand file modifications easily.
- Reduces manual effort involved in analyzing version differences and deployment-related changes.
- Supports automated rollback recommendations for restoring stable versions during de-ployment failures.
- Enables automated build, testing, and deployment workflows using Jenkins CI/CD pipelines.
- Ensures consistent deployment environments through Docker containerization.
- Provides real-time DevOps monitoring dashboards for tracking pipeline activity, deployment logs, and container health.
- Supports secure authentication, centralized monitoring, and scalable full stack architecture using React, Node.js, Express.js, and MongoDB.
- Improves deployment reliability, workflow efficiency, and operational visibility in software development environments.

5.2 Disadvantages

- The system requires technical knowledge for setup, deployment, and maintenance of DevOps tools and services.
- Initial configuration of Docker, Jenkins, MongoDB, and CI/CD pipelines may be complex for beginners.
- Continuous internet access is required for GitHub integration, AI services, and deployment synchronization.

- System performance depends on server resources, container management efficiency, and AI processing capabilities.
- Additional security configurations are required to protect APIs, containers, databases, and deployment services.
- AI-generated summaries may occasionally produce inaccurate or incomplete semantic interpretations for highly complex file changes.
- Regular maintenance and updates are necessary to manage dependencies, container images, and software compatibility.

5.3 Future Scope

The VersionIQ platform can be enhanced in the future by adding more advanced Artificial Intelligence and DevOps functionalities to improve deployment intelligence and workflow automation. Features such as conversational AI-based code assistants, automated root cause analysis, predictive deployment monitoring, and intelligent error detection can be integrated into the platform. These enhancements would provide deeper insights into version changes, deployment failures, and system performance.

The project can also be extended by deploying the platform on cloud environments such as AWS, Microsoft Azure, or Google Cloud to improve scalability, availability, and remote accessibility. Integration with Kubernetes can further enhance container orchestration, load balancing, and large-scale deployment management. Additional DevOps tools for monitoring, logging, and security analysis can also be implemented to strengthen system reliability and operational efficiency.

In the future, the platform may support multi-branch repository visualization, automated rollback deployment, and advanced analytics dashboards for tracking deployment trends and version intelligence. AI-powered recommendation systems can also be added to suggest deployment optimizations, detect risky modifications, and assist developers in understanding historical project evolution. These future enhancements can transform VersionIQ into a complete enterprise-level AI-powered Version Intelligence and DevOps Automation platform.